# AUTENTICAÇÃO, AUTORIZAÇÃO E JWT

# Node.js + Express + JSON Web Token

Material didático com explicação dos conceitos e de cada trecho de código.

## Objetivo do material

Ao final deste conteúdo, o aluno deverá compreender como uma API identifica usuários, protege rotas e controla permissões utilizando JWT (JSON Web Token).

# 1. Autenticação

**Autenticação** é o processo utilizado para verificar a identidade de uma pessoa ou sistema.

• **Pergunta respondida:** Quem é você?

• Exemplo: o usuário informa e-mail e senha.

• O servidor verifica se essas informações são válidas.

• Se estiverem corretas, o usuário é considerado autenticado.

**Exemplo prático:** quando você faz login em um sistema com e-mail e senha, está realizando uma autenticação.

# 2. Autorização

**Autorização** define o que um usuário autenticado pode fazer dentro do sistema.

• **Pergunta respondida:** O que você pode acessar ou fazer?

• Um usuário comum pode visualizar determinados dados.

• Um administrador pode cadastrar, editar ou excluir informações.

**Importante:** estar autenticado não significa automaticamente ter permissão para tudo.

## Autenticação × Autorização

| Autenticação | Autorização |
|---|---|
| Verifica quem é o usuário | Verifica o que ele pode fazer |
| Login | Permissões |
| Ex.: e-mail e senha | Ex.: ADMIN pode excluir usuários |

# 3. O que é JWT?

**JWT significa JSON Web Token.** É um padrão utilizado para criar tokens que podem representar a identidade e informações relacionadas ao usuário.

Depois que o usuário realiza o login com sucesso, o servidor pode gerar um token. Nas próximas requisições, o cliente envia esse token para provar sua identidade.

## Estrutura do JWT

```
HEADER.PAYLOAD.SIGNATURE
```

• **HEADER:** contém informações sobre o tipo do token e algoritmo.

• **PAYLOAD:** contém os dados definidos pela aplicação, como ID e função do usuário.

• **SIGNATURE:** ajuda a verificar se o token foi alterado.

**Atenção:** JWT não deve ser usado para armazenar informações secretas como senhas.

# 4. Bibliotecas utilizadas

```
npm init -y

npm install express jsonwebtoken bcrypt

npm install --save-dev nodemon
```

**Explicação:**

• **npm init -y:** cria o arquivo package.json utilizando configurações padrão.

• **express:** framework utilizado para criar o servidor e as rotas da API.

• **jsonwebtoken:** biblioteca utilizada para criar e validar tokens JWT.

• **bcrypt:** biblioteca utilizada para proteger senhas através de hash.

• **--save-dev nodemon:** instala o nodemon como dependência de desenvolvimento. Ele reinicia automaticamente o servidor quando o código é alterado.

# 5. Gerando um JWT

```
const jwt = require("jsonwebtoken");

const token = jwt.sign(
  {
    id: usuario.id,
    email: usuario.email,
    role: usuario.role
  },
  process.env.JWT_SECRET,
  { expiresIn: "1h" }
);

console.log(token);
```

## Explicação linha por linha

• **const jwt = require("jsonwebtoken");**
Importa a biblioteca jsonwebtoken e armazena suas funções na constante **jwt**.

• **const token = ...**
Cria uma constante chamada **token** para armazenar o JWT gerado.

• **jwt.sign(...)**
É a função responsável por criar e assinar o token.

• **id: usuario.id**
Coloca o identificador do usuário no payload.

• **email: usuario.email**
Inclui o e-mail do usuário no payload.

• **role: usuario.role**
Inclui a função/perfil do usuário, por exemplo ADMIN ou USER.

• **process.env.JWT_SECRET**
Utiliza uma variável de ambiente como chave secreta para assinar o token.

• **{ expiresIn: "1h" }**
Define que o token expira após uma hora.

• **console.log(token)**
Exibe o token gerado no terminal.

**Por que usar uma chave secreta?**

A chave secreta é utilizada para assinar o token. O servidor utiliza essa mesma chave posteriormente para verificar se o token é válido.

# 6. Enviando o JWT para a API

```
GET /usuarios HTTP/1.1

Authorization: Bearer SEU_TOKEN_AQUI
```

• **GET /usuarios:** representa uma requisição para buscar informações.

• **Authorization:** é o cabeçalho HTTP utilizado para enviar informações de autenticação.

• **Bearer:** indica que estamos enviando um token.

• **SEU_TOKEN_AQUI:** representa o JWT que foi gerado durante o login.

# 7. Middleware de autenticação

```
function autenticar(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader) {
    return res.status(401).json({
      mensagem: "Token não informado"
    });
  }

  const token = authHeader.split(" ")[1];

  try {
    const usuario = jwt.verify(
      token,
      process.env.JWT_SECRET
    );

    req.usuario = usuario;
    next();

  } catch (error) {
    return res.status(401).json({
      mensagem: "Token inválido"
    });
  }
}
```

## Explicação detalhada

• **function autenticar(req, res, next)**
Cria uma função middleware. Ela recebe a requisição (**req**), a resposta (**res**) e a função **next**.

• **req.headers.authorization**
Obtém o valor enviado no cabeçalho Authorization.

• **if (!authHeader)**
Verifica se o cabeçalho Authorization não foi enviado.

• **return res.status(401)**
Retorna o código HTTP 401 porque o usuário não apresentou uma autenticação válida.

• **authHeader.split(" ")[1]**
Separa o texto "Bearer TOKEN" usando o espaço e pega a segunda parte, que é o token.

• **try**
Inicia uma tentativa de validar o token.

• **jwt.verify(...)**
Verifica a assinatura e a validade do JWT.

• **req.usuario = usuario**
Armazena os dados obtidos do token dentro da requisição.

• **next()**
Permite que a execução continue para o próximo middleware ou para a rota.

• **catch (error)**
Captura erros, como token inválido, alterado ou expirado.

# 8. Criando uma rota protegida

```
app.get(
  "/perfil",
  autenticar,
  (req, res) => {
    res.json({
      mensagem: "Área protegida",
      usuario: req.usuario
    });
  }
);
```

## Explicação do código

• **app.get(...)**
Cria uma rota HTTP do tipo GET.

• **"/perfil"**
Define o endereço da rota.

• **autenticar**
Executa o middleware antes de permitir o acesso à rota.

• **(req, res) => {...}**
Função executada somente se a autenticação for bem-sucedida.

• **req.usuario**
Contém os dados que foram adicionados pelo middleware após validar o token.

• **res.json(...)**
Retorna uma resposta no formato JSON.

## Fluxo desta rota

• 1. Cliente faz uma requisição para /perfil.

• 2. O middleware autenticar verifica o token.

• 3. Se o token for válido, next() é chamado.

• 4. A rota /perfil é executada.

• 5. Se o token for inválido, a rota principal não é executada.

# 9. Middleware de autorização

```
function autorizar(...rolesPermitidas) {
  return (req, res, next) => {

    if (!rolesPermitidas.includes(req.usuario.role)) {
      return res.status(403).json({
        mensagem: "Acesso negado"
      });
    }

    next();
  };
}
```

## Explicação detalhada

**• function autorizar(...rolesPermitidas)**
Recebe uma quantidade variável de perfis permitidos. O operador **...** transforma os valores recebidos em uma lista.

**• return (req, res, next) =>**
Retorna um novo middleware que será utilizado nas rotas.

**• rolesPermitidas.includes(req.usuario.role)**
Verifica se o perfil do usuário existe na lista de perfis autorizados.

**• !**
Inverte o resultado. Se o perfil não estiver na lista, a condição será verdadeira.

**• status(403)**
Retorna o código 403: o usuário está autenticado, mas não possui permissão.

**• next()**
Permite continuar para a rota quando o usuário possui autorização.

# 10. Protegendo uma rota de administrador

```
app.delete(
  "/usuarios/:id",
  autenticar,
  autorizar("ADMIN"),
  (req, res) => {

    res.json({
      mensagem: "Usuário excluído!"
    });

  }
);
```

## Explicação passo a passo

**• app.delete(...)**
Cria uma rota HTTP para exclusão.

**• "/usuarios/:id"**
O :id representa um parâmetro da URL, normalmente o ID do usuário que será excluído.

**• autenticar**
Primeiro verifica se existe um usuário autenticado.

• **autorizar("ADMIN")**
Depois verifica se esse usuário possui o perfil ADMIN.

• **Somente depois** a função final da rota será executada.

• Essa ordem é importante: primeiro identificamos o usuário e depois verificamos suas permissões.

# 11. Diferença entre 401 e 403

| Código | Significado | Exemplo |
|---|---|---|
| 401 | Não autenticado / token inválido | Token não enviado ou expirado |
| 403 | Autenticado, mas sem permissão | USER tentando acessar área ADMIN |

# 12. Boas práticas de segurança

• **Nunca coloque senhas no JWT.**

• **Utilize uma chave secreta forte.**

• **Não deixe a chave secreta escrita diretamente no código.**

• **Utilize variáveis de ambiente.**

• **Defina um tempo de expiração para o token.**

• **Utilize HTTPS em produção.**

• **Armazene apenas informações necessárias no payload.**

• **Considere Refresh Tokens** em aplicações que precisam de sessões mais duradouras.

## Exemplo de variável de ambiente

```
JWT_SECRET=minha_chave_secreta_muito_forte
```

O arquivo com informações sensíveis normalmente não deve ser enviado para repositórios públicos.

# 13. Resumo final

• **Autenticação:** identifica quem é o usuário.

• **Autorização:** define o que o usuário pode fazer.

• **JWT:** permite transportar uma credencial assinada entre cliente e servidor.

• **Middleware:** permite interceptar e validar requisições.

• **Roles:** permitem controlar diferentes níveis de acesso.

# Desafio para os alunos ■

• Criar uma API utilizando Node.js e Express.

• Cadastrar usuários.

• Realizar login com e-mail e senha.

• Proteger a senha utilizando bcrypt.

• Gerar um JWT após o login.

• Criar um middleware para validar o token.

• Criar uma rota protegida.

• Criar dois perfis: USER e ADMIN.

• Permitir que apenas ADMIN execute uma rota de exclusão.

**Critério principal:** o aluno deve conseguir explicar o fluxo completo: login → geração do token → envio do token → autenticação → autorização.